CDC Shop – Architecture Decision Record

Conversation between Michaela Tóthová and Claude · 29 October 2026

MICHAELA

ROLE: You are an experienced software architect who writes clear Architecture Decision Records.

OBJECTIVE: Create an ADR for the CDC shop system based on my previous discussion about architectures.

SCENARIO: This is a university exercise. Attached are Task1_notes.md and the pdf file containing conversation from my previous session, where we compared 3 architectures (modular monolith, microservices with REST, event-driven) and chose microservices with REST (B). I will review the ADR and ask you to improve it a few times.

EXPECTED SOLUTION: An ADR in ADR_CDC.md. During the whole session, also keep short notes of our discussion in task2_notes.md: what I asked to change and why.

STEPS:
1. Read all attached files and create the notes file.
2. Write the first version of the ADR.
3. Wait for my feedback and update the ADR and notes after each change.

Attachments: Task1_notes.md, CDC_architecture_conversation_Task1.pdf

CLAUDE

I'll write the first ADR version from your Task 1 notes and conversation, plus the session notes file.

The first version of `ADR_CDC.md` records the decision for pure B with five services. It covers the context and decision drivers, the three considered options, the checkout flow, the rationale (including why the hybrid was dropped), and positive and negative consequences with the explicitly accepted trade-off. It also includes a mitigation table, the CDC gaps as open questions, and A as a fallback. `task2_notes.md` lists the inputs and summarises version 1, with an empty change log ready for your feedback.

One small correction compared to the Task 1 notes: the availability example now uses the three services that are actually critical in the final design (Order, Payment, Catalog → about 99.7 %), not the four from the earlier flow.

Files: ADR_CDC.md, task2_notes.md

MICHAELA

Stories 10 and 20 (About Us) are assigned to the Shipping/Notification service. That seems strange to me, isn't About Us just a static page in the frontend? Please fix that.

CLAUDE

You're right. About Us has no data or logic, so it doesn't belong in a backend service. In `ADR_CDC.md`, Shipping/Notification now covers only stories 9 and 23 (package info and the contact form). A sentence below the service table now says the About Us page is static content served by the frontend. The change and the reason for it are logged as Change 1 in `task2_notes.md`.

Files: ADR_CDC.md, task2_notes.md

MICHAELA

In the checkout flow, step 2 says Order checks "prices and availability" with Catalog, but in the open questions you say we don't know if stock is tracked. Isn't that a contradiction?

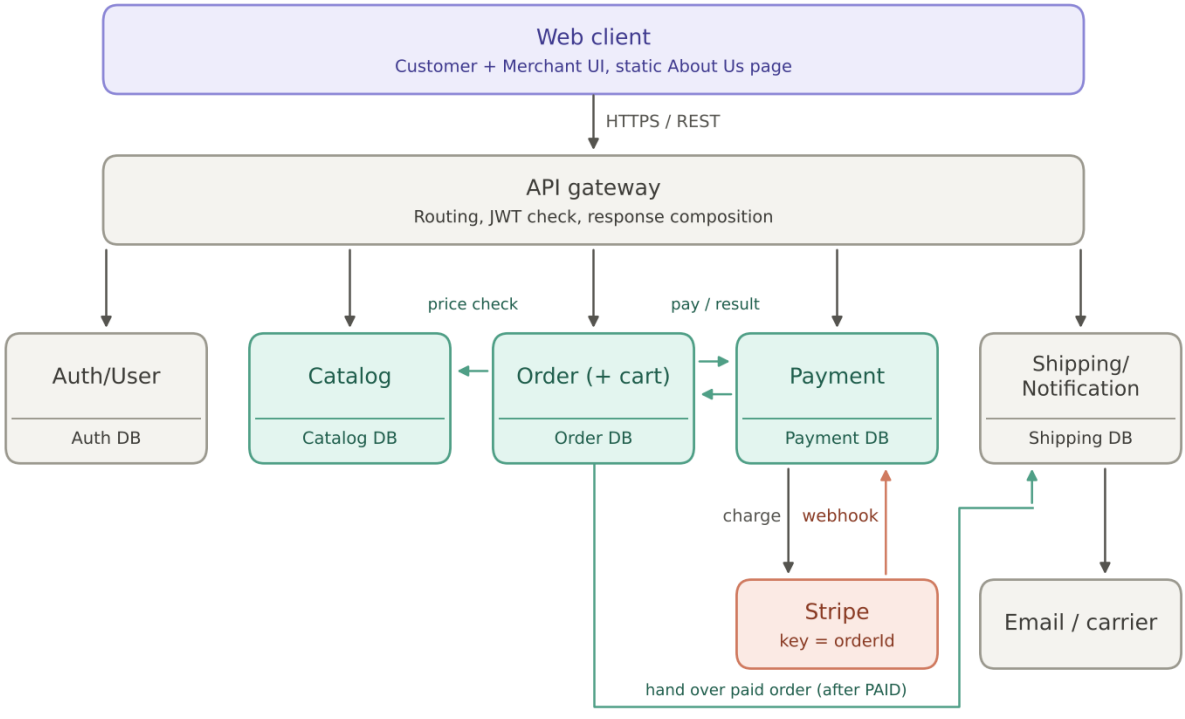
Story 9 is in both Order and Shipping/Notification. Who is actually responsible for what there?

And can you add a simple diagram of architecture B? Show the client, the API gateway, the services with their own databases, and Stripe.

CLAUDE

I've fixed all three points in `ADR_CDC.md`.

1. **Stock contradiction:** You're right, that was a contradiction. Step 2 now checks only the current prices and whether each product still exists, since a merchant may have removed it. It also says that if stock turns out to be tracked, the stock check goes in this step, and the open question about stock points back to it.
2. **Story 9:** Each service now owns its own part of the data. Order owns the order status (`PENDING_PAYMENT`, `PAID`, `FAILED`). Shipping/Notification owns the shipment data (carrier, tracking number, delivery status) once it receives the paid order. The tracking page gets both through the API gateway, which combines the two responses, so neither service keeps a copy of the other's data.
3. **Diagram:** The Decision section now has a plain-text diagram of the client, the gateway and the five services, each with its own database, plus Stripe and the email/carrier services. Below it is a short list of the four direct service-to-service calls, because the boxes alone don't show them. Drawing it showed a gap, so I also clarified step 5: Stripe's webhook goes to Payment, which then reports the result to Order.



Architecture B as added to ADR_CDC.md (drawn version of the plain-text diagram)

All three changes and the step 5 clarification are logged in `task2_notes.md`.

Files: ADR_CDC.md, task2_notes.md

MICHAELA

section 2 is still just a list. Could you turn it into a small comparison table of A, B and C, so it's clear at a glance why B won? And in the open questions there is "Does the teacher accept...". An ADR should be readable by anyone, so please phrase it without the teacher.

CLAUDE

I've made both changes in `ADR_CDC.md`.

Section 2 keeps a one-line description of each option and now has a comparison table. It compares A, B and C on whether they meet the "distributed" requirement, effort to build, operations, fit for the user stories, checkout consistency and behaviour when a service fails. The B column is in bold, and the last row gives a short verdict for each option. That row shows the reasoning in one line: A fails the main requirement, C meets it but is too complex, and B meets it at moderate cost.

Criterion	A – Modular monolith	B – Microservices (REST)	C – Event-driven
Meets "distributed"	Weakly, only by running copies	**Yes**, by splitting into services	Yes
Effort to build	Low	**Medium**	High
Operations	One app, one database	**Several services, gateway, logging**	Like B, plus a broker as critical infrastructure
Fit for the user stories (mostly simple requests)	Good	**Good**, request–response	Weaker, most stories don't need events
Checkout consistency	Local transactions; only Stripe needs care	**Coordinated by Order through API calls**	Eventual; outbox, idempotent consumers, saga compensations
When a service fails	Whole app is affected	**Failure stays in one service, but checkout needs Order, Payment and Catalog up**	Services can be briefly down; risk of duplicate, lost or reordered events
Verdict	Simplest, but fails the main driver	**Chosen**: meets the driver at moderate cost	Meets the driver, but too complex for this shop

Comparison table as added to section 2 of ADR_CDC.md

The open question no longer mentions the teacher. It now asks whether running several copies of one application (replication) is enough to meet the "distributed" requirement, and points to section 8.

Both changes are logged in `task2_notes.md` as Changes 5 and 6.

Files: ADR_CDC.md, task2_notes.md